

Inventario Detalhado da Pasta Source - Nyvorn

Projeto: Nyvorn

Pasta analisada: Nyvorn/Source

Total de arquivos .cs: 116

Objetivo do documento: listar cada arquivo .cs da pasta Source com uma descricao do que faz, suas dependencias principais e o ponto em que entra no fluxo do jogo. O foco e orientar navegacao de codigo, onboarding e planejamento de mudancas arquiteturais.

Como ler: cada entrada abaixo traz o caminho relativo do arquivo, a declaracao principal detectada, uma explicacao funcional, as dependencias mais relevantes encontradas no cabecalho do arquivo e a forma como aquele arquivo participa do fluxo de execucao do jogo.

Resumo por Grupo

- Data / Serialization / JsonLoader.cs: 1 arquivo(s)
- Engine / Content / AssetStore.cs: 1 arquivo(s)
- Engine / Debug / DebugDraw.cs: 1 arquivo(s)
- Engine / Generation / Noise: 2 arquivo(s)
- Engine / Graphics / Camera2D.cs: 1 arquivo(s)
- Engine / Input / InputService.cs: 1 arquivo(s)
- Engine / Input / InputState.cs: 1 arquivo(s)
- Engine / Physics / PhysicsSettings.cs: 1 arquivo(s)
- Engine / Physics / Sand: 1 arquivo(s)
- Game / StateMachine.cs: 1 arquivo(s)
- Game / States / DeathState.cs: 1 arquivo(s)
- Game / States / IGameState.cs: 1 arquivo(s)
- Game / States / InventoryState.cs: 1 arquivo(s)
- Game / States / LoadingWorldState.cs: 1 arquivo(s)
- Game / States / PauseMenuState.cs: 1 arquivo(s)
- Game / States / PlayingSession.cs: 1 arquivo(s)
- Game / States / PlayingSessionFactory.cs: 1 arquivo(s)
- Game / States / PlayingState.cs: 1 arquivo(s)
- Game / States / WorldCreationState.cs: 1 arquivo(s)
- Game / States / WorldEditState.cs: 1 arquivo(s)
- Game / States / WorldSelectState.cs: 1 arquivo(s)
- Gameplay / Combat / CombatSystem.cs: 1 arquivo(s)
- Gameplay / Combat / DamageSystem.cs: 1 arquivo(s)
- Gameplay / Combat / Interfaces: 2 arquivo(s)
- Gameplay / Combat / Resolvers: 2 arquivo(s)
- Gameplay / Combat / Weapons: 3 arquivo(s)
- Gameplay / Entities / Enemies: 8 arquivo(s)
- Gameplay / Entities / Player: 9 arquivo(s)
- Gameplay / Items / Hotbar.cs: 1 arquivo(s)
- Gameplay / Items / Inventory.cs: 1 arquivo(s)

- Gameplay / Items / InventorySlot.cs: 1 arquivo(s)
- Gameplay / Items / ItemDefinition.cs: 1 arquivo(s)
- Gameplay / Items / ItemDefinitions.cs: 1 arquivo(s)
- Gameplay / Items / ItemId.cs: 1 arquivo(s)
- Gameplay / Items / WorldItem.cs: 1 arquivo(s)
- Gameplay / UI / ElyraSkyRenderer.cs: 1 arquivo(s)
- Gameplay / UI / HudRenderer.cs: 1 arquivo(s)
- Gameplay / UI / TextLayout.cs: 1 arquivo(s)
- Gameplay / UI / WorldHealthBarRenderer.cs: 1 arquivo(s)
- Gameplay / UI / WorldMinimapRenderer.cs: 1 arquivo(s)
- Gameplay / UI / WorldTilePreviewRenderer.cs: 1 arquivo(s)
- Gameplay / World / Generation: 23 arquivo(s)
- Gameplay / World / Persistence: 9 arquivo(s)
- Gameplay / World / Simulation: 7 arquivo(s)
- Gameplay / World / Tile.cs: 1 arquivo(s)
- Gameplay / World / TileItemMapper.cs: 1 arquivo(s)
- Gameplay / World / Tissue: 12 arquivo(s)
- Gameplay / World / WorldChunkCoord.cs: 1 arquivo(s)
- Gameplay / World / WorldMap.cs: 1 arquivo(s)

Data / Serialization / JsonLoader.cs

Data/Serialization/JsonLoader.cs

Declaracao principal: declaracao principal nao detectada rapidamente

O que faz: Carrega e desserializa dados externos usados pelo jogo.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra quando algum subsistema precisa transformar dados serializados em estruturas usadas pelo jogo.

Engine / Content / AssetStore.cs

Engine/Content/AssetStore.cs

Declaracao principal: declaracao principal nao detectada rapidamente

O que faz: Centraliza acesso e reaproveitamento de assets carregados.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: E um utilitario de infraestrutura usado indiretamente por varios pontos do fluxo do jogo.

Engine / Debug / DebugDraw.cs

Engine/Debug/DebugDraw.cs

Declaracao principal: declaracao principal nao detectada rapidamente

O que faz: Fornece desenho e apoio visual para depuracao em runtime.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: E um utilitario de infraestrutura usado indiretamente por varios pontos do fluxo do jogo.

Engine / Generation / Noise

Engine/Generation/Noise/FastNoiseLite.cs

Declaracao principal: class FastNoiseLite

O que faz: Implementa algoritmos de ruido procedural reutilizados pela worldgen.

Dependencias principais: FNLfloat = System.Single

Onde entra no fluxo do jogo: E um utilitario de infraestrutura usado indiretamente por varios pontos do fluxo do jogo.

Engine/Generation/Noise/OpenSimplex2F.cs

Declaracao principal: class OpenSimplex2F

O que faz: Implementa algoritmos de ruido procedural reutilizados pela worldgen.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: E um utilitario de infraestrutura usado indiretamente por varios pontos do fluxo do jogo.

Engine / Graphics / Camera2D.cs

Engine/Graphics/Camera2D.cs

Declaracao principal: class Camera2D

O que faz: Controla a camera 2D, conversoes de espaco e comportamento de seguimento.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra durante o gameplay e no draw, definindo enquadramento, conversao tela/mundo e comportamento de camera.

Engine / Input / InputService.cs

Engine/Input/InputService.cs

Declaracao principal: class InputService

O que faz: Le input bruto do MonoGame e o converte em um estado consistente para o gameplay.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Input

Onde entra no fluxo do jogo: Entra no inicio do frame, antes do gameplay, quando o input bruto e convertido para um estado consumivel por PlayingState e PlayingSession.

Engine / Input / InputState.cs

Engine/Input/InputState.cs

Declaracao principal: struct InputState

O que faz: Representa o snapshot consolidado de input consumido pela simulacao de jogo.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra no inicio do frame, antes do gameplay, quando o input bruto e convertido para um estado consumivel por PlayingState e PlayingSession.

Engine / Physics / PhysicsSettings.cs

Engine/Physics/PhysicsSettings.cs

Declaracao principal: class PhysicsSettings

O que faz: Expose constantes globais de fisica como gravidade e ajustes compartilhados.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: E um utilitario de infraestrutura usado indiretamente por varios pontos do fluxo do jogo.

Engine / Physics / Sand

Engine/Physics/Sand/SandSystem.cs

Declaracao principal: class SandSystem

O que faz: Simula areia em nível de pixel usando ocupação espacial e lista ativa de partículas.

Dependências principais: World, Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra na simulação do mundo durante o gameplay; hoje participa do ciclo da sessão e da simulação discreta/ativa de areia.

Game / StateMachine.cs

Game/StateMachine.cs

Declaração principal: class StateMachine

O que faz: Gerencia a pilha de estados do jogo e controla a ordem de update e draw.

Dependências principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Game.States

Onde entra no fluxo do jogo: Fica logo abaixo de Game1 e decide quais estados recebem Update e Draw em cada frame.

Game / States / DeathState.cs

Game/States/DeathState.cs

Declaração principal: class DeathState

O que faz: Apresenta a tela de morte e oferece reinício da sessão.

Dependências principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Content, Microsoft.Xna.Framework.Graphics, Microsoft.Xna.Framework.Input

Onde entra no fluxo do jogo: Entra como parte da navegação entre telas, carregamento ou menus do jogo.

Game / States / IGameState.cs

Game/States/IGameState.cs

Declaração principal: interface IGameState

O que faz: Define o contrato comum que todos os estados do jogo precisam seguir.

Dependências principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics

Onde entra no fluxo do jogo: Entra como parte da navegação entre telas, carregamento ou menus do jogo.

Game / States / InventoryState.cs

Game/States/InventoryState.cs

Declaracao principal: class InventoryState

O que faz: Abre e controla a interacao com o inventario durante o gameplay.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Microsoft.Xna.Framework.Input, Game, Gameplay.Items

Onde entra no fluxo do jogo: Entra como parte da navegacao entre telas, carregamento ou menus do jogo.

Game / States / LoadingWorldState.cs

Game/States/LoadingWorldState.cs

Declaracao principal: class LoadingWorldState

O que faz: Executa e exibe o progresso de criacao ou carregamento de uma sessao.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Content, Microsoft.Xna.Framework.Graphics, Gameplay.UI

Onde entra no fluxo do jogo: Entra como parte da navegacao entre telas, carregamento ou menus do jogo.

Game / States / PauseMenuState.cs

Game/States/PauseMenuState.cs

Declaracao principal: class PauseMenuState

O que faz: Congela o gameplay e mostra o menu de pausa.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Content, Microsoft.Xna.Framework.Graphics, Microsoft.Xna.Framework.Input, Game, Gameplay.UI

Onde entra no fluxo do jogo: Entra como parte da navegacao entre telas, carregamento ou menus do jogo.

Game / States / PlayingSession.cs

Game/States/PlayingSession.cs

Declaracao principal: class PlayingSession

O que faz: Orquestra os sistemas centrais da sessao jogavel: mundo, player, inimigos, itens, UI e simulacao.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Engine.Input, Engine.Graphics, Engine.Physics.Sand, Gameplay.Combat

Onde entra no fluxo do jogo: Fica no centro do gameplay em runtime e coordena a maior parte da logica de simulacao e interacao do mundo.

Game / States / PlayingSessionFactory.cs

Game/States/PlayingSessionFactory.cs

Declaracao principal: class PlayingSessionFactory

O que faz: Monta uma sessao completa carregando assets, saves, mundo e entidades.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Content, Microsoft.Xna.Framework.Graphics, Engine.Graphics, Gameplay.Combat, Gameplay.Combat.Weapons

Onde entra no fluxo do jogo: Entra quando um mundo e criado ou carregado, montando a sessao antes do estado jogavel assumir.

Game / States / PlayingState.cs

Game/States/PlayingState.cs

Declaracao principal: class PlayingState

O que faz: Estado principal jogavel que conecta input, update, camera, HUD e render do mundo.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Content, Microsoft.Xna.Framework.Graphics, Microsoft.Xna.Framework.Input, Engine.Input, Game

Onde entra no fluxo do jogo: E o ponto principal do fluxo de jogo jogavel: recebe input, atualiza a sessao e dispara o render do mundo.

Game / States / WorldCreationState.cs

Game/States/WorldCreationState.cs

Declaracao principal: class WorldCreationState

O que faz: Tela para configurar e criar um novo mundo.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Content, Microsoft.Xna.Framework.Graphics, Microsoft.Xna.Framework.Input, Gameplay.UI, World.Generation

Onde entra no fluxo do jogo: Entra como parte da navegacao entre telas, carregamento ou menus do jogo.

Game / States / WorldEditState.cs

Game/States/WorldEditState.cs

Declaracao principal: class WorldEditState

O que faz: Tela de edicao simples para renomear ou ajustar um mundo salvo.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Content, Microsoft.Xna.Framework.Graphics, Microsoft.Xna.Framework.Input, Gameplay.UI, World.Generation

Onde entra no fluxo do jogo: Entra como parte da navegacao entre telas, carregamento ou menus do jogo.

Game / States / WorldSelectState.cs

Game/States/WorldSelectState.cs

Declaracao principal: class WorldSelectState

O que faz: Tela inicial de navegacao e selecao dos mundos salvos.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Content, Microsoft.Xna.Framework.Graphics, Microsoft.Xna.Framework.Input, Gameplay.UI, World.Generation

Onde entra no fluxo do jogo: Entra como parte da navegacao entre telas, carregamento ou menus do jogo.

Gameplay / Combat / CombatSystem.cs

Gameplay/Combat/CombatSystem.cs

Declaracao principal: class CombatSystem

O que faz: Coordena as resolucoes de combate entre fontes de dano e alvos.

Dependencias principais: Gameplay.Combat.Interfaces, Gameplay.Combat.Resolvers

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay / Combat / DamageSystem.cs

Gameplay/Combat/DamageSystem.cs

Declaracao principal: declaracao principal nao detectada rapidamente

O que faz: Agrupa regras de dano e possiveis utilitarios de aplicacao.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay / Combat / Interfaces

Gameplay/Combat/Interfaces/IDamageable.cs

Declaracao principal: interface IDamageable

O que faz: Contrato para objetos capazes de receber dano.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay/Combat/Interfaces/IHitSource.cs

Declaracao principal: interface IHitSource

O que faz: Contrato para objetos que fornecem hitbox e dados de ataque.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay / Combat / Resolvers

Gameplay/Combat/Resolvers/EnemyContactResolver.cs

Declaracao principal: class EnemyContactResolver

O que faz: Resolve dano por contato entre inimigos e alvos.

Dependencias principais: Gameplay.Combat.Interfaces

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay/Combat/Resolvers/PlayerAttackResolver.cs

Declaracao principal: class PlayerAttackResolver

O que faz: Resolve os ataques ativos do jogador contra alvos validos.

Dependencias principais: Gameplay.Combat.Interfaces

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay / Combat / Weapons

Gameplay/Combat/Weapons/HandWeapon.cs

Declaracao principal: class HandWeapon

O que faz: Implementa uma arma corpo a corpo base de mao.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, World

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay/Combat/Weapons/Pickaxe.cs

Declaracao principal: class Pickaxe

O que faz: Implementa a picareta com comportamento de arma e ferramenta de mineracao.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Gameplay.Entities.Player, World

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay/Combat/Weapons/Weapon.cs

Declaracao principal: class Weapon

O que faz: Classe base para armas, frames e regras de ataque/equipamento.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Gameplay.Entities.Player, World

Onde entra no fluxo do jogo: Entra durante a atualizacao do gameplay quando ataques, hitboxes e aplicacao de dano precisam ser resolvidos.

Gameplay / Entities / Enemies

Gameplay/Entities/Enemies/Enemy.cs

Declaracao principal: class Enemy

O que faz: Representa a entidade inimiga completa em runtime.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Gameplay.Combat.Interfaces, Gameplay.Entities.Enemies.EnemyAnimations, World

Onde entra no fluxo do jogo: Entra na sessao jogavel quando inimigos proximos sao simulados, atacam, recebem dano e sao desenhados.

Gameplay/Entities/Enemies/EnemyAnimations/EnemyTestAnimations.cs

Declaracao principal: class EnemyTestAnimations

O que faz: Define os clips de animacao do inimigo de teste usado no projeto.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra na sessao jogavel quando inimigos proximos sao simulados, atacam, recebem dano e sao desenhados.

Gameplay/Entities/Enemies/EnemyAnimator.cs

Declaracao principal: class EnemyAnimator

O que faz: Controla o estado visual e a troca de frames dos inimigos.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra na sessao jogavel quando inimigos proximos sao simulados, atacam, recebem dano e sao desenhados.

Gameplay/Entities/Enemies/EnemyAnimState.cs

Declaracao principal: enum EnemyAnimState

O que faz: Enumera os estados de animacao possiveis do inimigo.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra na sessao jogavel quando inimigos proximos sao simulados, atacam, recebem dano e sao desenhados.

Gameplay/Entities/Enemies/EnemyCombat.cs

Declaracao principal: class EnemyCombat

O que faz: Mantem timers e reacoes de combate do inimigo.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra na sessao jogavel quando inimigos proximos sao simulados, atacam, recebem dano e sao desenhados.

Gameplay/Entities/Enemies/EnemyConfig.cs

Declaracao principal: class EnemyConfig

O que faz: Centraliza atributos, danos e configuracoes do inimigo.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra na sessao jogavel quando inimigos proximos sao simulados, atacam, recebem dano e sao desenhados.

Gameplay/Entities/Enemies/EnemyMotor.cs

Declaracao principal: class EnemyMotor

O que faz: Executa movimento, gravidade e colisao do inimigo no mundo.

Dependencias principais: Microsoft.Xna.Framework, Engine.Physics, World

Onde entra no fluxo do jogo: Entra na sessao jogavel quando inimigos proximos sao simulados, atacam, recebem dano e sao desenhados.

Gameplay/Entities/Enemies/EnemyRespawnController.cs

Declaracao principal: class EnemyRespawnController

O que faz: Controla reaparecimento de inimigos apos derrota.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics

Onde entra no fluxo do jogo: Entra na sessao jogavel quando inimigos proximos sao simulados, atacam, recebem dano e sao desenhados.

Gameplay / Entities / Player

Gameplay/Entities/Player/Animation.cs

Declaracao principal: class Animation

O que faz: Clip de animacao generico baseado em frames com duracao.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay/Entities/Player/AnimationState.cs

Declaracao principal: enum AnimationState

O que faz: Lista os estados visuais usados pelas animacoes do player.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay/Entities/Player/AnimFrame.cs

Declaracao principal: struct AnimFrame

O que faz: Estrutura de frame individual com source rect e duracao.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay/Entities/Player/Player.cs

Declaracao principal: class Player

O que faz: Representa o jogador e conecta input, combate, motor e animacao.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Engine.Physics.Sand, Engine.Input, Gameplay.Combat.Interfaces, Gameplay.Combat.Weapons

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay/Entities/Player/PlayerAnimation.cs

Declaracao principal: class PlayerAnimations

O que faz: Agrupa definicoes de clips e composicoes visuais do player.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay/Entities/Player/PlayerAnimator.cs

Declaracao principal: class PlayerAnimator

O que faz: Gerencia poses, direcao, draw layers e transicoes visuais do jogador.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay/Entities/Player/PlayerCombat.cs

Declaracao principal: class PlayerCombat

O que faz: Controla ataques, dodge, hitbox, dano recebido e cooldowns do jogador.

Dependencias principais: Microsoft.Xna.Framework, Gameplay.Combat.Weapons

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay/Entities/Player/PlayerConfig.cs

Declaracao principal: class PlayerConfig

O que faz: Concentra numeros de tuning do jogador: movimento, interacao e combate.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay/Entities/Player/PlayerMotor.cs

Declaracao principal: class PlayerMotor

O que faz: Executa fisica, deslocamento, gravidade, step-up e colisao do jogador.

Dependencias principais: Microsoft.Xna.Framework, Engine.Physics, Engine.Physics.Sand, World

Onde entra no fluxo do jogo: Entra em praticamente todo frame jogavel, porque o jogador e atualizado, animado e desenhado continuamente.

Gameplay / Items / Hotbar.cs

Gameplay/Items/Hotbar.cs

Declaracao principal: class Hotbar

O que faz: Inventario rapido de acesso imediato usado pelo jogador.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra quando o jogo precisa equipar, armazenar, soltar, coletar ou renderizar itens no mundo e na UI.

Gameplay / Items / Inventory.cs

Gameplay/Items/Inventory.cs

Declaracao principal: class Inventory

O que faz: Container geral de slots para itens e armazenamento.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra quando o jogo precisa equipar, armazenar, soltar, coletar ou renderizar itens no mundo e na UI.

Gameplay / Items / InventorySlot.cs

Gameplay/Items/InventorySlot.cs

Declaracao principal: class InventorySlot

O que faz: Representa um slot individual com item e quantidade.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra quando o jogo precisa equipar, armazenar, soltar, coletar ou renderizar itens no mundo e na UI.

Gameplay / Items / ItemDefinition.cs

Gameplay/Items/ItemDefinition.cs

Declaracao principal: class ItemDefinition

O que faz: Define propriedades de um item do jogo.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra quando o jogo precisa equipar, armazenar, soltar, coletar ou renderizar itens no mundo e na UI.

Gameplay / Items / ItemDefinitions.cs

Gameplay/Items/ItemDefinitions.cs

Declaracao principal: class ItemDefinitions

O que faz: Catalogo estatico com todas as definicoes de itens.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra quando o jogo precisa equipar, armazenar, soltar, coletar ou renderizar itens no mundo e na UI.

Gameplay / Items / ItemId.cs

Gameplay/Items/ItemId.cs

Declaracao principal: enum ItemId

O que faz: Enum ou identificador central dos itens conhecidos.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra quando o jogo precisa equipar, armazenar, soltar, coletar ou renderizar itens no mundo e na UI.

Gameplay / Items / WorldItem.cs

Gameplay/Items/WorldItem.cs

Declaracao principal: class WorldItem

O que faz: Item solto no mundo com fisica, pickup delay e renderizacao.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Engine.Physics, World

Onde entra no fluxo do jogo: Entra quando o jogo precisa equipar, armazenar, soltar, coletar ou renderizar itens no mundo e na UI.

Gameplay / UI / ElyraSkyRenderer.cs

Gameplay/UI/ElyraSkyRenderer.cs

Declaracao principal: class ElyraSkyRenderer

O que faz: Desenha o céu e a atmosfera de fundo do mundo.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics

Onde entra no fluxo do jogo: Entra no draw do gameplay e, em alguns casos, também no processamento de interação de interface como minimapa e inventário.

Gameplay / UI / HudRenderer.cs

Gameplay/UI/HudRenderer.cs

Declaracao principal: class HudRenderer

O que faz: Renderiza HUD, hotbar, vida e painel de inventário.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Gameplay.Items

Onde entra no fluxo do jogo: Entra no draw do gameplay e, em alguns casos, também no processamento de interação de interface como minimapa e inventário.

Gameplay / UI / TextLayout.cs

Gameplay/UI/TextLayout.cs

Declaracao principal: class TextLayout

O que faz: Quebra e organiza texto para caber em caixas da interface.

Dependencias principais: Microsoft.Xna.Framework.Graphics

Onde entra no fluxo do jogo: Entra no draw do gameplay e, em alguns casos, também no processamento de interação de interface como minimapa e inventário.

Gameplay / UI / WorldHealthBarRenderer.cs

Gameplay/UI/WorldHealthBarRenderer.cs

Declaracao principal: class WorldHealthBarRenderer

O que faz: Desenha barras de vida sobre entidades do mundo.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics

Onde entra no fluxo do jogo: Entra no draw do gameplay e, em alguns casos, tambem no processamento de interacao de interface como minimapa e inventario.

Gameplay / UI / WorldMinimapRenderer.cs

Gameplay/UI/WorldMinimapRenderer.cs

Declaracao principal: record struct

O que faz: Desenha o minimapa e processa sua interacao.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, Engine.Graphics, World, World.Generation, World.Tissue

Onde entra no fluxo do jogo: Entra no draw do gameplay e, em alguns casos, tambem no processamento de interacao de interface como minimapa e inventario.

Gameplay / UI / WorldTilePreviewRenderer.cs

Gameplay/UI/WorldTilePreviewRenderer.cs

Declaracao principal: enum WorldTilePreviewState

O que faz: Mostra o preview visual para colocar ou quebrar tiles.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics

Onde entra no fluxo do jogo: Entra no draw do gameplay e, em alguns casos, tambem no processamento de interacao de interface como minimapa e inventario.

Gameplay / World / Generation

Gameplay/World/Generation/IWorldGenPass.cs

Declaracao principal: interface IWorldGenPass

O que faz: Contrato para passos individuais da geracao procedural.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/OpenSimplexNoise.cs

Declaracao principal: class OpenSimplexNoise

O que faz: Fornece ruido procedural para a pipeline de geracao.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/Passes/BaseTerrainFillPass.cs

Declaracao principal: class BaseTerrainFillPass

O que faz: Preenche a massa base do mundo com material solido inicial antes dos refinamentos seguintes.

Dependencias principais: Gameplay.World.Simulation

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/Passes/CaveEntrancePass.cs

Declaracao principal: class CaveEntrancePass

O que faz: Abre entradas naturais que conectam superficie e cavernas.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/Passes/CavePass.cs

Declaracao principal: class CavePass

O que faz: Esculpe cavernas e vazios subterraneos a partir de ruido e regras de profundidade.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

[Gameplay/World/Generation/Passes/ClearWorldPass.cs](#)

Declaracao principal: class ClearWorldPass

O que faz: Limpa ou reinicializa o mapa antes de outros passos de geracao.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

[Gameplay/World/Generation/Passes/DirtToStoneTransitionPass.cs](#)

Declaracao principal: class DirtToStoneTransitionPass

O que faz: Define a transicao gradual entre terra e pedra conforme a profundidade.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

[Gameplay/World/Generation/Passes/LayerBoundaryPass.cs](#)

Declaracao principal: class LayerBoundaryPass

O que faz: Ajusta fronteiras entre camadas geologicas e zonas do mundo.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

[Gameplay/World/Generation/Passes/SurfaceProfilePass.cs](#)

Declaracao principal: class SurfaceProfilePass

O que faz: Modela o contorno da superficie e a silhueta geral do terreno.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/Passes/TissueField.cs

Declaracao principal: class TissueField

O que faz: Executa um passo especifico da pipeline de geracao do mundo.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/Passes/TissuePass.cs

Declaracao principal: class TissuePass

O que faz: Distribui tecido subterraneo durante a geracao do planeta.

Dependencias principais: Microsoft.Xna.Framework, World.Generation

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/Passes/WorldBoundsPass.cs

Declaracao principal: class WorldBoundsPass

O que faz: Aplica limites do mundo e garante bordas consistentes para a simulacao.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldFieldSampler.cs

Declaracao principal: class WorldFieldSampler

O que faz: Faz amostragens auxiliares sobre campos usados na geracao.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldGenConfig.cs

Declaracao principal: class WorldGenConfig

O que faz: Guarda configuracoes e parametros da geracao procedural.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldGenContext.cs

Declaracao principal: class WorldGenContext

O que faz: Compartilha estado entre os passos da geracao do mundo.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldGenDebugOptions.cs

Declaracao principal: class WorldGenDebugOptions

O que faz: Controla opcoes de diagnostico da worldgen.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldGenerator.cs

Declaracao principal: class WorldGenerator

O que faz: Orquestra a sequencia de passos que constroem um mundo.

Dependencias principais: Microsoft.Xna.Framework, World.Generation.Passes

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldGenPhaseDefinition.cs

Declaracao principal: class WorldGenPhaseDefinition

O que faz: Descreve cada fase de geracao e seu peso/progresso.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldGenProgressReporter.cs

Declaracao principal: class WorldGenProgressReporter

O que faz: Expose progresso e mensagens da geracao em andamento.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldLayerDefinition.cs

Declaracao principal: struct WorldLayerDefinition

O que faz: Descreve camadas verticais e suas propriedades.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldLayerType.cs

Declaracao principal: enum WorldLayerType

O que faz: Enumera os tipos de camada do mundo.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldNoiseSampler.cs

Declaracao principal: class WorldNoiseSampler

O que faz: Faz leituras padronizadas de ruido para regras geograficas.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay/World/Generation/WorldSizePreset.cs

Declaracao principal: enum WorldSizePreset

O que faz: Define presets de tamanho para novos mundos.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra durante criacao ou regeneracao de mundos, antes de a sessao jogavel ficar disponivel ao player.

Gameplay / World / Persistence

Gameplay/World/Persistence/PlanetSaveData.cs

Declaracao principal: class PlanetSaveData

O que faz: Modelo completo de persistencia de um mundo salvo.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay/World/Persistence/PlanetSaveService.cs

Declaracao principal: class PlanetSaveService

O que faz: Le e grava saves completos dos mundos.

Dependencias principais: Game.States, Gameplay.Items

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay/World/Persistence/PlanetSaveSummary.cs

Declaracao principal: class PlanetSaveSummary

O que faz: Resumo leve de um save usado na listagem de mundos.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay/World/Persistence/PlanetWorldMetadata.cs

Declaracao principal: class PlanetWorldMetadata

O que faz: Metadados centrais de identificacao e configuracao de um mundo.

Dependencias principais: World.Generation

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay/World/Persistence/PlayerInventorySlotSaveData.cs

Declaracao principal: class PlayerInventorySlotSaveData

O que faz: Representa um slot persistido do inventario do jogador.

Dependencias principais: Gameplay.Items

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay/World/Persistence/PlayerSaveData.cs

Declaracao principal: class PlayerSaveData

O que faz: Modelo de persistencia do estado do jogador.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay/World/Persistence/PlayerSaveService.cs

Declaracao principal: class PlayerSaveService

O que faz: Le e grava o progresso persistido do jogador.

Dependencias principais: Game.States, Gameplay.Items

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay/World/Persistence/WorldItemSaveData.cs

Declaracao principal: class WorldItemSaveData

O que faz: Persistencia de itens soltos no mundo.

Dependencias principais: Gameplay.Items

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay/World/Persistence/WorldTileChange.cs

Declaracao principal: record struct

O que faz: Representa alteracoes persistentes em tiles apos a geracao.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra ao salvar, carregar, resumir ou restaurar estado persistente de mundos e do jogador.

Gameplay / World / Simulation

Gameplay/World/Simulation/ActiveSimulationChunkSelector.cs

Declaracao principal: class ActiveSimulationChunkSelector

O que faz: Calcula quais chunks devem entrar na simulacao ativa perto do jogador/camera.

Dependencias principais: Microsoft.Xna.Framework, World

Onde entra no fluxo do jogo: Entra na camada de simulacao do gameplay, especialmente no scheduler de ticks e em atualizacoes discretas do mundo.

Gameplay/World/Simulation/GrassSimulation.cs

Declaracao principal: class GrassSimulation

O que faz: Executa a simulacao discreta de propagacao da grama.

Dependencias principais: World

Onde entra no fluxo do jogo: Entra na camada de simulacao do gameplay, especialmente no scheduler de ticks e em atualizacoes discretas do mundo.

[Gameplay/World/Simulation/RandomTileUpdateHelper.cs](#)

Declaracao principal: class RandomTileUpdateHelper

O que faz: Fornece sorteio controlado de tiles para updates aleatorios em chunks ativos.

Dependencias principais: Microsoft.Xna.Framework, World

Onde entra no fluxo do jogo: Entra na camada de simulacao do gameplay, especialmente no scheduler de ticks e em atualizacoes discretas do mundo.

[Gameplay/World/Simulation/WorldTickConfig.cs](#)

Declaracao principal: class WorldTickConfig

O que faz: Configura as taxas e limites do scheduler de world ticks.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra na camada de simulacao do gameplay, especialmente no scheduler de ticks e em atualizacoes discretas do mundo.

[Gameplay/World/Simulation/WorldTickDispatch.cs](#)

Declaracao principal: record struct

O que faz: Representa quantos ticks de cada faixa foram disparados em um passo.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra na camada de simulacao do gameplay, especialmente no scheduler de ticks e em atualizacoes discretas do mundo.

[Gameplay/World/Simulation/WorldTickSnapshot.cs](#)

Declaracao principal: record struct

O que faz: Snapshot de debug do estado atual do scheduler de ticks.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra na camada de simulacao do gameplay, especialmente no scheduler de ticks e em atualizacoes discretas do mundo.

Gameplay/World/Simulation/WorldTickSystem.cs

Declaracao principal: class WorldTickSystem

O que faz: Acumula tempo e dispara FastTick, MediumTick e SlowTick.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra na camada de simulacao do gameplay, especialmente no scheduler de ticks e em atualizacoes discretas do mundo.

Gameplay / World / Tile.cs

Gameplay/World/Tile.cs

Declaracao principal: enum TileType

O que faz: Representa o conceito base de tile do mapa.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabecalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra no ciclo principal do mundo, sustentando tiles, chunks, consultas espaciais e alteracoes persistentes.

Gameplay / World / TileItemMapper.cs

Gameplay/World/TileItemMapper.cs

Declaracao principal: class TileItemMapper

O que faz: Converte entre tipos de tile e itens correspondentes.

Dependencias principais: Gameplay.Items

Onde entra no fluxo do jogo: Entra no ciclo principal do mundo, sustentando tiles, chunks, consultas espaciais e alteracoes persistentes.

Gameplay / World / Tissue

Gameplay/World/Tissue/TissueAnalysisResult.cs

Declaracao principal: class TissueAnalysisResult

O que faz: Armazena o resultado analitico da malha de tecido.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueAnalyzer.cs

Declaracao principal: class TissueAnalyzer

O que faz: Analisa o campo de tecido e extrai hubs, links e classificacoes.

Dependencias principais: Microsoft.Xna.Framework, World.Generation, static World.Tissue.TissueLink

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueBranch.cs

Declaracao principal: class TissueBranch

O que faz: Representa um ramo da estrutura de tecido.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueDebugPalette.cs

Declaracao principal: class TissueDebugPalette

O que faz: Define cores de debug para visualizar o tecido.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueFieldDebugRenderer.cs

Declaracao principal: class TissueFieldDebugRenderer

O que faz: Desenha a visualizacao de debug do tecido no mundo.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, World, World.Generation

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueGenerator.cs

Declaracao principal: class TissueGenerator

O que faz: Gera proceduralmente a rede de tecido subterranea.

Dependencias principais: Microsoft.Xna.Framework, World.Generation

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueHub.cs

Declaracao principal: class TissueHub

O que faz: Representa um hub importante da rede de tecido.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueLink.cs

Declaracao principal: class TissueLink

O que faz: Representa uma conexao entre hubs de tecido.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueLocalType.cs

Declaracao principal: enum TissueLocalType

O que faz: Classifica o tipo local de tecido em uma area.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueNetwork.cs

Declaracao principal: class TissueNetwork

O que faz: Estrutura agregada da rede de tecido gerada.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueNode.cs

Declaracao principal: class TissueNode

O que faz: Representa um no individual da rede de tecido.

Dependencias principais: Microsoft.Xna.Framework

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay/World/Tissue/TissueRevealController.cs

Declaracao principal: class TissueRevealController

O que faz: Controla o efeito de revelar e destacar tecido em runtime.

Dependencias principais: Microsoft.Xna.Framework, Engine.Input

Onde entra no fluxo do jogo: Entra tanto na geracao quanto no gameplay, especialmente em analise, debug, reveal e regras ligadas ao tecido.

Gameplay / World / WorldChunkCoord.cs

Gameplay/World/WorldChunkCoord.cs

Declaracao principal: record struct

O que faz: Representa uma coordenada logica de chunk.

Dependencias principais: Dependencias locais relevantes nao inferidas apenas pelo cabeçalho; o arquivo depende principalmente de tipos basicos ou do proprio subsistema.

Onde entra no fluxo do jogo: Entra no ciclo principal do mundo, sustentando tiles, chunks, consultas espaciais e alteracoes persistentes.

Gameplay / World / WorldMap.cs

Gameplay/World/WorldMap.cs

Declaracao principal: class WorldMap

O que faz: Mantem a grade de tiles, caches, alteracoes e regras principais do mundo.

Dependencias principais: Microsoft.Xna.Framework, Microsoft.Xna.Framework.Graphics, World.Tissue, World.Generation, World.Persistence

Onde entra no fluxo do jogo: Entra no ciclo principal do mundo, sustentando tiles, chunks, consultas espaciais e alteracoes persistentes.